
Student Performance Prediction System

Project Id: AIML-5

Name: Devraj Singh Rawat (Team Leader)

Class Roll Number: 38

University Roll Number: 2028707

Section and Group: U(G2)

Name: Astik Tripathi

Class Roll Number: 35

University Roll Number: 2028678

Section and Group: U(G2)

Name: Mukul Singh Bhandari

Class Roll Number: 49

University Roll Number: 2028773

Section and Group: U(G2)

TABLE OF CONTENTS

1. Problem Description	3
1.1. Dataset Description.....	3
1.2. Flow diagram.....	4
2. Modules for Project Implementation.....	5
2.1. Modules Use	
2.2. Platform Used	6
3. Machine Learning model/ Searching Technique used.....	7
4. Evaluation metrics used	8
5. Results and Visualizations	9
6. Conclusion and Future Scope	11
7. References.....	12

1. Problem Description

Road traffic accidents are a major global concern, causing significant loss of life, injuries, and property damage every year. Predicting the severity of accidents is an important task that can help authorities, emergency services, and transportation planners take preventive measures and improve road safety. However, accident severity depends on multiple factors such as vehicle speed, weather conditions, road conditions, and vehicle type, making it difficult to determine the potential impact of an accident using traditional analysis methods.

The aim of the Traffic Accident Severity Prediction system is to use machine learning techniques to analyze historical accident data and predict the severity level of future accidents. By processing and analyzing relevant features from accident datasets, the system can identify patterns and relationships between different factors that contribute to accident outcomes. Data preprocessing is performed to clean the dataset by removing irrelevant or missing records, ensuring better model performance. Feature selection is used to identify the most influential variables affecting accident severity.

Machine learning algorithms such as Logistic Regression and Support Vector Machine (SVM) with different kernels are applied to build predictive models. The performance of these models is evaluated using metrics such as confusion matrix and recall score. This predictive approach can help in improving traffic management, enhancing road safety strategies, and supporting faster emergency response systems.

1.1 Dataset Description

The dataset used in this project is **synthetically generated**, designed to simulate real-world traffic accident conditions based on the structure and characteristics of the original US-Accidents dataset. It consists of **8,400 rows (instances)** and **27 features (variables)** in total. To better reflect real-world data collection scenarios, **controlled noise** has been intentionally introduced into the dataset, simulating inconsistencies, measurement errors, and variability that are commonly observed in actual traffic accident records.

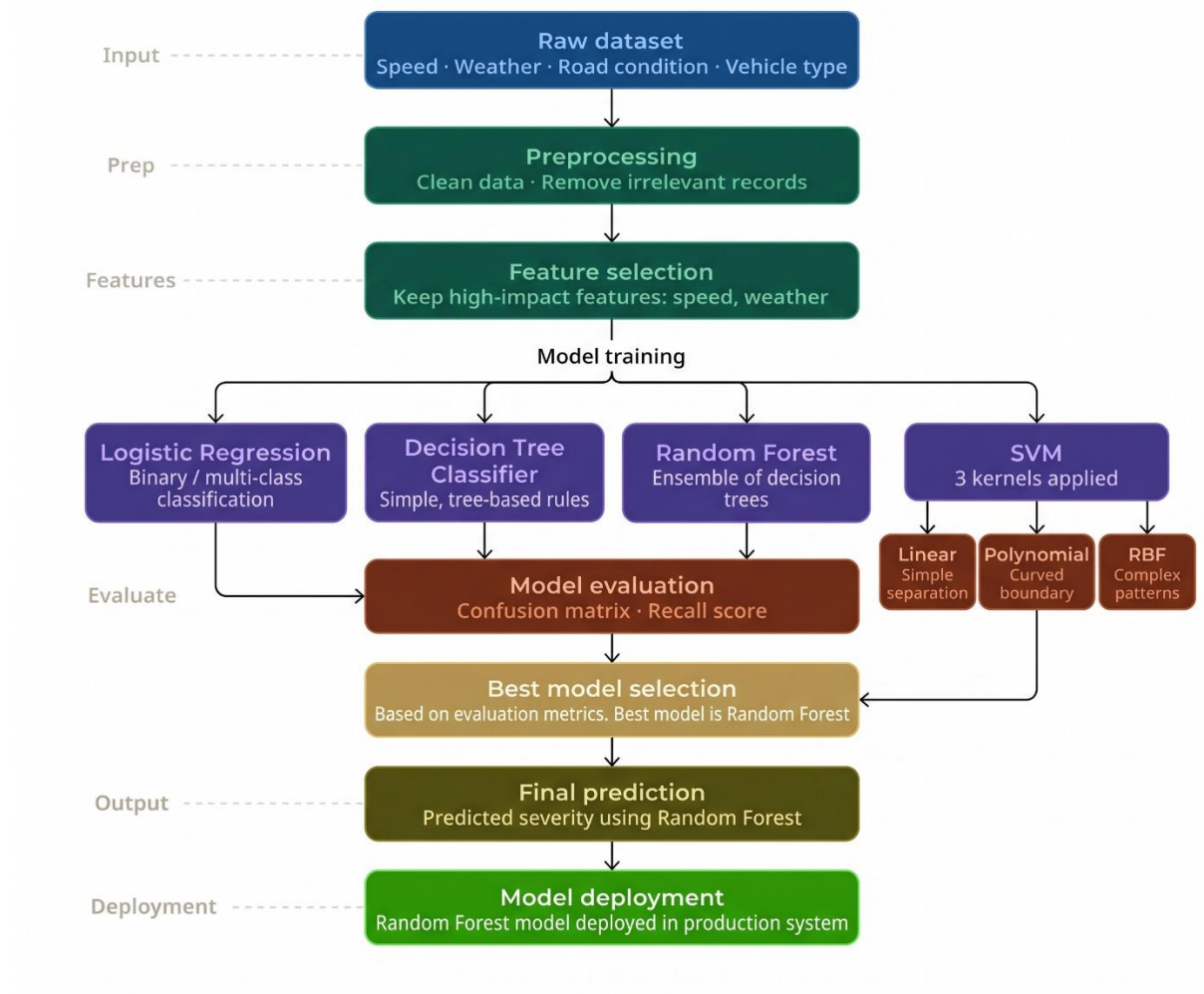
Of the 27 features, **8 are identified as primary/main features**, representing the core attributes that most directly influence the target outcome. The remaining 19 features serve as supplementary variables, capturing additional contextual or derived information.

Key characteristics of the dataset:

- **Type:** Synthetic with controlled noise (based on US-Accidents structure)
- **Total Instances:** 8,400 rows
- **Total Features:** 27
- **Primary Features:** 8 (*e.g., weather_condition, road_condition, speed_kmh, alcohol_involved, visibility_m, light_condition, driver_experience_yrs, accident_severity*)
- **Supplementary Features:** 19

The inclusion of noise ensures that the dataset presents realistic challenges for model training and evaluation, such as handling outliers, imperfect measurements, and slight label inconsistencies. This makes the dataset a more robust and practical benchmark for assessing model performance under conditions that closely resemble real-world scenarios.

1.2 Flow Diagram



Flow chart

2. Modules for Project Implementation

2.1 Modules Used(Inbuilt/ Custom)

i. Pandas (pandas)

Purpose: Pandas has been used for data manipulation, analysis, and preprocessing of the accident dataset.

Functions Used:

- `pd.read_csv()` – To read the traffic accident dataset from a CSV file into a Pandas DataFrame.
- `df.head()` – To display the first few rows of the dataset for initial inspection.
- `df.tail()` – To display the last few rows of the dataset.
- `df.shape` – To retrieve the dimensions (rows and columns) of the dataset.
- `df.info()` – To display a summary of the DataFrame including data types and non-null counts.
- `df.describe()` – To generate descriptive statistics such as mean, standard deviation, and percentile values.
- `df.isnull()` – To detect missing values in the dataset.
- `df.isnull().sum()` – To count the total number of missing values in each column.
- `df.dropna()` – To remove rows containing missing/null values during data cleaning.
- `df.drop()` – To remove irrelevant or redundant columns from the dataset.
- `df['column'].value_counts()` – To count the frequency of unique values in a column (used for target variable analysis).
- `df.fillna()` – To fill missing values with a specified value or strategy.
- `df.groupby()` – To group data based on categorical features for aggregation and analysis.
- `df.rename()` – To rename columns for clarity.
- `pd.get_dummies()` – To perform one-hot encoding on categorical features.

ii. NumPy (numpy)

Purpose: NumPy has been used for numerical computations, array operations, and mathematical functions required during data preprocessing and model evaluation.

Functions Used:

- `np.array()` – To convert data into NumPy arrays for model input.

- `np.mean()` – To calculate the mean of numerical columns.
- `np.unique()` – To find unique class labels in the target variable.

iii. Matplotlib (matplotlib.pyplot)

Purpose: Matplotlib has been used for data visualization, including plotting graphs and charts to understand the distribution and relationships in the dataset.

Functions Used:

- `plt.figure()` – To create a new figure for plotting.
- `plt.plot()` – To draw line plots for trend analysis.
- `plt.bar()` / `plt.barh()` – To create bar charts for categorical feature distribution.
- `plt.hist()` – To plot histograms for numerical feature distribution.
- `plt.pie()` – To create pie charts for class/severity distribution.
- `plt.scatter()` – To create scatter plots for visualizing relationships between features.
- `plt.xlabel()` / `plt.ylabel()` – To label the X and Y axes of plots.
- `plt.title()` – To add a title to the plot.
- `plt.legend()` – To add a legend to differentiate plot elements.
- `plt.show()` – To display the generated plot.
- `plt.savefig()` – To save the plot as an image file.
- `plt.tight_layout()` – To automatically adjust subplot spacing for clean layout.

iv. Seaborn (seaborn)

Purpose: Seaborn has been used for advanced statistical data visualization, providing more aesthetically informative plots built on top of Matplotlib.

Functions Used:

- `sns.heatmap()` – To visualize the correlation matrix and the confusion matrix as a heatmap.
- `sns.countplot()` – To plot the count distribution of categorical features such as Severity levels.
- `sns.boxplot()` – To identify outliers and visualize the distribution of numerical features.
- `sns.barplot()` – To display the mean/aggregated values of features grouped by severity.
- `sns.pairplot()` – To plot pairwise relationships between features in the dataset.

- `sns.histplot()` – To plot the distribution of a numerical variable with optional KDE overlay.
- `sns.set_style()` – To set the aesthetic style of the plots.

v. Scikit-learn (sklearn)

Purpose: Scikit-learn is the primary machine learning library used for data preprocessing, model building, training, and evaluation of the Traffic Accident Severity Prediction system.

a. sklearn.model_selection

- `train_test_split()` – To split the dataset into training and testing sets (typically 80:20 or 70:30 ratio).
- `cross_val_score()` – To evaluate model performance using k-fold cross-validation.

b. sklearn.preprocessing

- `LabelEncoder()` – To convert categorical target labels (severity levels) into numerical format.
- `StandardScaler()` – To standardize feature values by removing the mean and scaling to unit variance, which is essential for SVM.
- `MinMaxScaler()` – To normalize feature values to a fixed range [0, 1] for consistent model input.

c. sklearn.linear_model

- `LogisticRegression()` – To build the Logistic Regression classification model for predicting accident severity.
 - `.fit()` – To train the Logistic Regression model on the training data.
 - `.predict()` – To generate severity predictions on the test data.
 - `.predict_proba()` – To obtain class probability scores for each prediction.
 - `.score()` – To compute the accuracy of the trained model.

d. sklearn.svm

- `SVC()` (Support Vector Classifier) – To build the SVM model with various kernel functions (linear, polynomial, RBF, sigmoid) for accident severity classification.
 - `.fit()` – To train the SVM model using the training data.
 - `.predict()` – To predict accident severity levels on the test data.
 - `.score()` – To evaluate the accuracy of the SVM model.

e. sklearn.metrics

- `confusion_matrix()` – To generate the confusion matrix showing true vs. predicted severity class counts.
 - `recall_score()` – To calculate the recall (sensitivity) of the model per class.
 - `accuracy_score()` – To compute the overall prediction accuracy of the model.
 - `precision_score()` – To calculate the precision of the model.
 - `f1_score()` – To compute the F1-score as a harmonic mean of precision and recall.
 - `classification_report()` – To display a complete performance report including precision, recall, and F1-score for each class.
- f. `sklearn.feature_selection`
- `SelectKBest()` – To select the top K most relevant features based on statistical tests.
 - `chi2()` – To apply the Chi-Squared test for feature selection among categorical variables.
 - `f_classif()` – To apply ANOVA F-test for selecting the most significant numerical features.

vi. Warnings (warnings)

Purpose: The warnings module has been used to suppress unnecessary warning messages during model training and library operations, keeping the output clean and readable.

Functions Used:

- `warnings.filterwarnings('ignore')` – To suppress all runtime or deprecation warnings during execution.

2.2 Platform Used:(mention all the platforms with the reason of using it)

I Visual Studio Code (VS Code)

- **Purpose:** For code writing, testing, and debugging.
- VS Code is a lightweight yet powerful source code editor developed by Microsoft. It has been used as the primary code editor for writing Python scripts related to data preprocessing, feature selection, and model building. VS Code is equipped with multiple extensions such as the **Python Extension**, **IntelliSense** (for code auto-completion), an integrated **debugger** (for identifying and resolving errors in the code), and a built-in **terminal** (for running Python scripts directly without switching applications). The **Pylint** and **Flake8** extensions further assist in maintaining code quality and style consistency throughout the project.
- **Key Features Used:**
 - i. Syntax highlighting and code auto-completion via IntelliSense
 - ii. Integrated terminal for running Python scripts
 - iii. Debugging tools for identifying logical and runtime errors
 - iv. Extension support for Jupyter Notebooks within VS Code
 - v. Git integration for version control

II Jupyter Notebook

- **Purpose:** For interactive code execution, data exploration, and result visualization.
- Jupyter Notebook is an open-source, web-based interactive computing environment that allows code, visualizations, and narrative text to be combined in a single document called a **notebook** (.ipynb file). It has been extensively used in this project for **Exploratory Data Analysis (EDA)**, stepwise data preprocessing, model training, and displaying evaluation metrics and plots in a clean, readable format.
- Jupyter Notebook is particularly well-suited for machine learning projects because it allows each step — such as loading data, cleaning it, training the model, and evaluating results — to be executed **cell by cell**, making it easy to inspect intermediate outputs and iteratively refine the approach without re-running the entire program.
- **Key Features Used:**
 - i. Cell-by-cell execution for step-by-step code testing and debugging
 - ii. Inline visualization of Matplotlib and Seaborn plots directly within the notebook

- iii. Markdown support for adding explanations, headings, and documentation alongside code
- iv. Easy display of Pandas DataFrames in tabular format for data inspection
- v. Support for exporting the notebook as HTML, PDF, or Python script for reporting purposes
- vi. Kernel management for selecting and managing the Python environment

III Anaconda Distribution

- **Purpose:** For managing Python environments and installing data science libraries.
- Anaconda is an open-source distribution of Python specifically designed for data science and machine learning. It has been used to set up and manage the Python environment for this project, providing easy installation of all required libraries such as NumPy, Pandas, Scikit-learn, Matplotlib, and Seaborn through its package manager **conda**. Anaconda also comes pre-installed with Jupyter Notebook, making it a convenient all-in-one platform for data science development.
- **Key Features Used:**
 - i. conda package manager for installing and managing Python libraries
 - ii. Pre-installed data science libraries (NumPy, Pandas, Scikit-learn, etc.)
 - iii. Virtual environment creation to isolate project dependencies
 - iv. Anaconda Navigator for a graphical interface to launch Jupyter Notebook and manage packages

IV GitHub (Version Control)

- **Purpose:** For source code version control and project collaboration.
- GitHub is a cloud-based platform built on top of Git for version control and collaborative software development. It has been used in this project to maintain version history of the code, track changes made during different stages of development (data preprocessing, model training, evaluation), and serve as a backup repository for the project files.
- **Key Features Used:**
 - i. git init, git add, git commit – For tracking and saving changes to the codebase
 - ii. git push / git pull – For uploading and synchronizing code

with the remote repository

- iii. Repository management for organizing project files and notebooks
- iv. Commit history for reviewing and reverting changes when needed

3. Machine Learning Model/ Searching Technique Used

The models used in this project are Support Vector Machine (SVM) and Logistic Regression. These are supervised machine learning algorithms commonly used for classification tasks and were selected for their accuracy and reliability.

Support Vector Machine (SVM)

SVM is used to classify data by finding the optimal boundary, called a hyperplane, that separates classes with maximum margin. Different kernels were applied in this project:

- **Linear SVM:** Suitable for linearly separable data using a straight-line boundary.
- **Polynomial SVM:** Handles non-linear data by creating curved decision boundaries.
- **RBF (Radial Basis Function) SVM:** Effective for complex datasets with highly non-linear patterns by mapping data into higher dimensions.
-

Logistic Regression

Logistic Regression is a supervised learning algorithm used for binary and multi-class classification problems. It predicts the probability of an input belonging to a particular class using the sigmoid function. It is simple, efficient, and widely used for classification tasks.

Random Forest

Random Forest is an ensemble learning algorithm that combines multiple decision trees to improve classification accuracy and reduce overfitting. It works by taking predictions from many trees and selecting the final result through majority voting. It is robust and performs well on large datasets.

Grid Search CV

Grid Search CV is a hyperparameter tuning technique used to find the best combination of model parameters. It uses cross-validation to test multiple parameter values and selects the combination that gives the highest performance. It helps in optimizing models like SVM, Logistic Regression, and Random Forest.

4. Evaluation metrics used

To measure the performance of the machine learning models, different evaluation metrics were used. These metrics help in understanding the accuracy and effectiveness of the models.

- **Accuracy Score:** Measures the overall percentage of correctly predicted instances out of total predictions. It is one of the most common metrics for classification tasks.
- **Precision:** Measures how many of the predicted positive cases were actually positive. It is important when false positives need to be minimized.
- **Recall:** Measures how many actual positive cases were correctly identified by the model. It is useful when false negatives need to be reduced.
- **F1-Score:** The harmonic mean of precision and recall. It provides a balanced measure when both false positives and false negatives are important.
- **Confusion Matrix:** A table used to visualize model performance by showing correct and incorrect predictions in terms of True Positive, True Negative, False Positive, and False Negative values.

5. Results and Visualizations

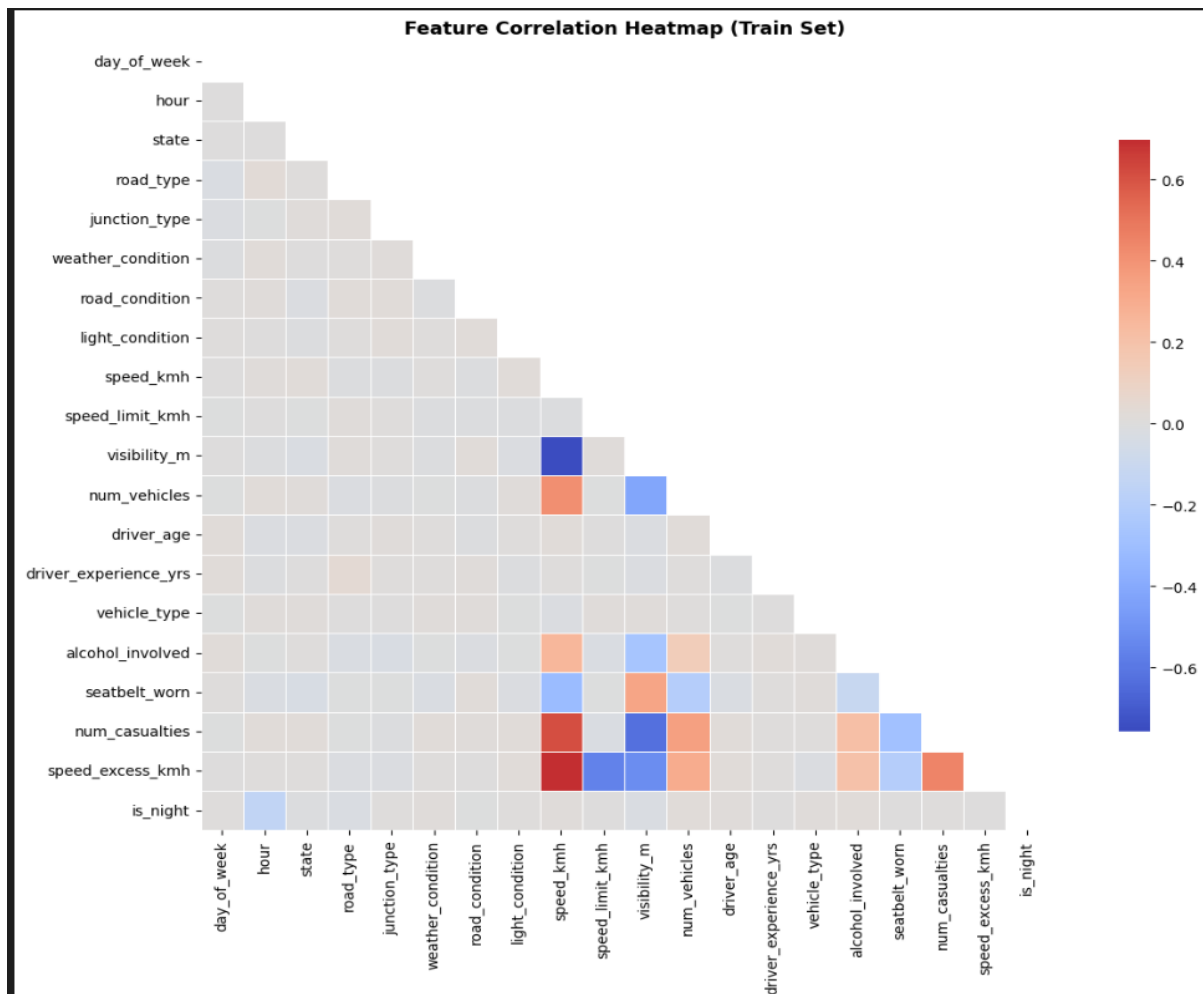


Fig.2 Heatmap showing co-relation.

Logistic Regression				
	precision	recall	f1-score	support
Minor	0.94	0.95	0.95	534
Moderate	0.91	0.91	0.91	533
Severe	0.97	0.96	0.97	533
accuracy			0.94	1600
macro avg	0.94	0.94	0.94	1600
weighted avg	0.94	0.94	0.94	1600

Fig.3 Classification Report(Logistic Regression)

SVM Linear				
	precision	recall	f1-score	support
Minor	0.95	0.95	0.95	534
Moderate	0.91	0.92	0.92	533
Severe	0.97	0.96	0.97	533
accuracy			0.94	1600
macro avg	0.94	0.94	0.94	1600
weighted avg	0.94	0.94	0.94	1600

Fig.4 Classification Report(SVM - Linear)

SVM Polynomial				
	precision	recall	f1-score	support
Minor	0.96	0.94	0.95	534
Moderate	0.88	0.94	0.91	533
Severe	0.98	0.93	0.96	533
accuracy			0.94	1600
macro avg	0.94	0.94	0.94	1600
weighted avg	0.94	0.94	0.94	1600

Fig.5 Classification Report(SVM - Polynomial)

SVM RBF				
	precision	recall	f1-score	support
Minor	0.96	0.95	0.95	534
Moderate	0.91	0.93	0.92	533
Severe	0.98	0.96	0.97	533
accuracy			0.95	1600
macro avg	0.95	0.95	0.95	1600
weighted avg	0.95	0.95	0.95	1600

Fig.6 Classification Report(SVM - RBF)

----- Random Forest -----				
	precision	recall	f1-score	support
Minor	0.96	0.97	0.97	534
Moderate	0.95	0.94	0.94	533
Severe	0.97	0.98	0.98	533
accuracy			0.96	1600
macro avg	0.96	0.96	0.96	1600
weighted avg	0.96	0.96	0.96	1600

Fig.7 Classification Report(Random Forest)

```

PS D:\P R O J E C T S\Traffic Accident Severity Prediction\FINAL> python predict.py

Select model:
 1. Logistic Regression
 2. SVM Linear
 3. SVM Polynomial
 4. SVM RBF
 5. Random Forest
 6. All (majority vote)

Enter choice (1-6): 2

Features: ['speed_kmh', 'visibility_m', 'num_vehicles', 'alcohol_involved', 'seatbelt_worn', 'num_casualties', 'speed_excess_kmh', 'is_night']
Enter values as comma-separated list: 80, 500, 2, 0, 1, 1, 20, 0

=====
C:\Users\astik\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\utils\validation.py:2749: UserWarning:
X does not have valid feature names, but SVC was fitted with feature names
  warnings.warn(
SVM Linear          → Minor
=====

```

Fig.8 Final Output

6. Conclusion and Future Scope:

Conclusion:

The project was successfully implemented using Python and various libraries for data analysis and machine learning. The system efficiently processes the dataset, performs necessary preprocessing, and generates meaningful insights/predictions. Visualization techniques helped in better understanding of patterns in the data. Overall, the project demonstrates how machine learning can be applied to solve real-world problems with satisfactory performance.

Future Scope:

- Improve model accuracy by using advanced algorithms and hyperparameter tuning.
- Use larger and real-time datasets for better results.
- Deploy the model as a web application for real-world usage.
- Integrate deep learning techniques for more complex predictions.
- Enhance user interface and automation for better usability.

7. References:

Books:

Let Us Python – Yashavant Kanetkar

Online Resources:

GeeksforGeeks – <https://www.geeksforgeeks.org/>

GitHub – <https://github.com/>

Scikit-learn Documentation — <https://scikit-learn.org/>

Python Official Docs — <https://docs.python.org/>